

Introduction to Application GUIs & Image Integration

Assignment 3

Deadline: 2nd July EOD

Instructions

- Attempt all questions.
 - Write clean, readable and modular code.
 - Write theoretical answers as comments (with proper numbering) at the beginning of the corresponding .py file.
 - Use comments wherever necessary.
 - Only the **NumPy** and **Pillow** libraries are permitted unless explicitly mentioned.
 - Submit the solution for **each question separately**. Do not combine multiple questions into a single Python file.
 - Organize your solution locally before uploading it to GitHub.
-

Submission Guidelines

Repository

https://github.com/SHUB-1806/CubeCanvas_Speedcubing

Your submission must be placed inside

Assignments/Assignment3/<Your Name>/

with the following folder structure.

```
<Your_Name>/
|
|-- Q1/
|-- Q2/
|-- Q3/
|-- Q4/
\-- Mini_Project/
```

Each folder should contain only the files required for that particular question.

Question	Expected Files
Q1	solution.py
Q2	solution.py, output images (if any)
Q3	solution.py, generated images
Q4	solution.py, comparison images
Mini Project	project.py + supporting files

Important: Do **not** upload every file individually to the repository root. Keep your work organized locally and then upload your complete folder structure.

Git Workflow

Follow the steps below to submit your assignment.

1. Clone the repository

```
git clone https://github.com/SHUB-1806/CubeCanvas_Speedcubing.git
cd CubeCanvas_Speedcubing
```

2. Create your submission directory

```
Assignments/
  Assignment3/
    <Your_Name>/
```

3. Copy all your solution folders into this directory.

4. Verify your changes

```
git status
```

5. Stage, commit and push

```
git add Assignments/Assignment3/<Your_Name>
```

```
git commit -m "Assignment 3 Submission - <Your_Name>"
```

```
git push origin main
```

6. Open GitHub in your browser and create a Pull Request.

Use the following format:

Name -- Roll Number -- Assignment 3 Submission

Example:

Shubham Joshi -- 240500 -- Assignment 3 Submission

Unable to Use Git?

If you are unable to use Git, create the same folder structure directly on GitHub under

Assignments/Assignment3/

and upload each file to its corresponding directory. Avoid uploading all files directly inside the Assignment3 folder.

Unable to do anything?

DM me

Question 1: Grid Geometry and Layouts (Easy)

In modern GUI development, making an application responsive to window resizing is critical.

Part A (Theory) Answer the following:

1. What is the purpose of the `weight` parameter in `grid_rowconfigure` and `grid_columnconfigure`?
2. What does the argument `sticky="nsew"` do when placing a widget?
3. Explain the concept of “stacking” frames and how the `tkraise()` method works.

Part B (Coding) Write a standalone Python script using CustomTkinter that:

1. Creates a main window of 400x400.
2. Divides the window perfectly into a 2×2 grid (4 equal quadrants).
3. Places a differently colored `CTkFrame` in each quadrant.
4. Ensures that when the window is maximized, all four quadrants stretch equally to fill the space.

Submission Requirements for Q1:

- `solution.py` containing code and the theoretical answers as comments at the top
-

Question 2: Event Binding and Callbacks (Medium)

GUIs are “event-driven,” meaning they wait for user interactions (events) before executing code.

Part A (Theory) Answer the following:

1. What is an event binding in Tkinter?
2. Explain the difference between triggering a function via a button’s `command` parameter versus using `.bind("<KeyRelease>", function)`.
3. Why sometimes we store function as `function()` and sometimes `function` in command? What’s the difference?
4. Why do we sometimes need to use `lambda` functions when assigning commands to buttons or bindings?

Part B (Coding) Create a simple CustomTkinter window with:

1. A text entry box (`CTkEntry`).
2. A label (`CTkLabel`) below it starting with the text “Character Count: 0”.
3. Bind an event to the entry box so that every time a key is pressed and released, the label updates in real-time to show the exact number of characters currently typed in the box.

Submission Requirements for Q2:

- `solution.py` containing your code and the theoretical answers as comments at the top.
 - Any output screenshots/images.
-

Question 3: Aspect Ratio and Memory Management (Medium)

Integrating Pillow (PIL) with Tkinter requires careful handling of dimensions and memory.

Part A (Theory) Answer the following:

1. In proportional image scaling, why do we calculate the scale factor using `min(frame_width / img_width, frame_height / img_height)` and not `max()`?
2. What is Python’s Garbage Collection, and why must we explicitly store a reference to `ImageTk.PhotoImage` (e.g., `label.image = tk_image`) to prevent our image from disappearing?

Part B (Coding) Write a function `safe_resize(image_path, max_w, max_h)` that:

1. Opens an image using Pillow.
2. Calculates the new dimensions without distorting the aspect ratio, ensuring it fits entirely within `max_w` and `max_h`.
3. Resizes the image using Lanczos resampling.

4. Returns the *dimensions* (width, height) of the newly resized image as a tuple, also displaying the image dynamically fit to window.

Submission Requirements for Q3:

- `solution.py` containing your function, test code, and the theoretical answers as comments at the top.
 - short screen-recording showing the working of the resize function
-

Question 4 (Mini Project)

Interactive WCA ID Card Generator

Create a standalone GUI application using CustomTkinter and Pillow that allows a user to design a digital World Cube Association (WCA) ID card. Your application must have a two-column responsive layout to manage the data inputs and visualize the front and back of the card.

Interface Requirements:

1. **Left Panel (Input Form):**
 - A button to “Upload 1x1 Photo” (opens a file dialog to select a portrait image).
 - Text entry fields for: **Name** (e.g., Max Park) and **WCA ID** (e.g., 2012PARK03).
 - A text entry field for **Status** (e.g., MEMBER, DELEGATE) and a hex color code entry to define the status box background color.
 - A checklist to list **Participating Events** (e.g., 3x3, OH, 4x4) to be displayed on the back.
2. **Right Panel (Live ID Preview):**
 - Two distinct display frames acting as the canvases for the **Front** and **Back** of the ID Card.
 - **Front Canvas:** Display the uploaded portrait. Below or beside the portrait, dynamically display the Name, WCA ID, and the Status text layered over the chosen hex background color.
 - **Back Canvas:** Display the string of Participating Events and a placeholder static graphic representing a Barcode.

Behavioral Requirements:

1. **Responsiveness:** If the user stretches the main window, the Right Panel’s canvases should trigger a `<Configure>` event and dynamically resize themselves to fit the new available space.
2. **Live Updates:** Typing into the Name, WCA ID, Status, or Events entry boxes should instantly update the text labels on the ID preview using `<KeyRelease>` bindings.
3. **Error Handling:** If a user types an invalid hex code into the status color entry, the app should safely ignore it rather than crashing.

Bonus Features (Optional) Implement any two:

1. Add a “Save ID” button that captures the final Front and Back frame layouts and exports them as PNG files.
2. Implement a script to automatically generate a real Code 128 or QR barcode image based on the typed WCA ID and place it on the back canvas.
3. Add a default placeholder silhouette image that shows up on the front canvas before the user uploads a portrait.

Submission Requirements for Mini Project:

- **Source Code:** `project.py` and any other modular supporting `.py` files.
- **Visual Proof (Crucial):** A screen recording (`.mp4` or `.gif`) demonstrating:
 - The application window being resized (proving `<Configure>` scaling works).
 - Real-time text updating on the ID card as you type (proving `<KeyRelease>` bindings).
 - Typing an invalid hex code to show the application safely ignores it.

Note: If you cannot capture a video, provide at least 4 distinct screenshots covering these states (Initial launch, fully populated ID, maximized window, and invalid hex entry attempt).
- **Exported Artifacts:** At least one generated ID card (`.png`) if the “Save ID” bonus was attempted.

Example Output

A sample ID card is shown below. Your final output does not need to match this design exactly, but it should contain all the required information and be visually well-organized.

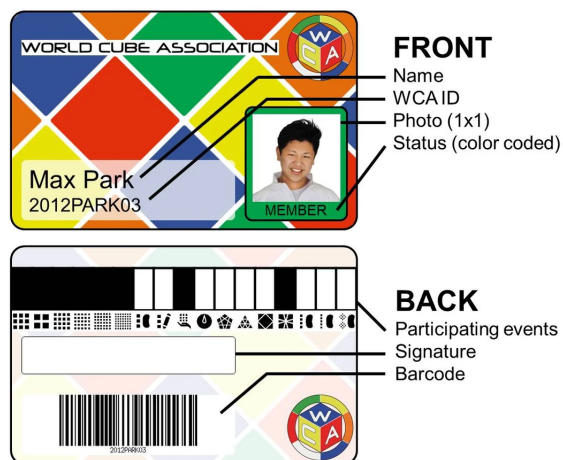


Figure 1: Sample ID Card